



O mesmo Spark. Sem a JVM no caminho quente.

Apache DataFusion Comet troca os operadores físicos do Spark por execução nativa em Rust e Arrow. Nenhuma linha do pipeline muda — só quatro configurações. Aqui estão a arquitetura e o ganho **medido**, rodando no Sparquet Studio.

Spark 4.1.1

Comet 1.0.0

40 milhões de linhas

2,52× na agregação

POR QUE EXISTE

O plano é ótimo. A execução é JVM.

- O Catalyst decide **o que** fazer muito bem. Quem faz é código Java gerado em tempo de execução.
- Parquet é **colunar**; boa parte do Spark processa **linha a linha**, com objeto, boxing e GC no meio.
- Vetorização de verdade — SIMD, lotes de Arrow — não é algo que a JVM entregue de graça.
- Escalar isso custa nó, e nó custa fatura. O gargalo não é o disco: é CPU gasta em overhead.

Comet não substitui o Spark. Ele mantém Catalyst, API, catálogo e ecossistema — e troca **só o operador físico** por uma implementação nativa.

O QUE É

Um plugin do Spark, não um motor novo.

Comet é um projeto da Apache Software Foundation construído sobre o **DataFusion** — motor de consulta em Rust, colunar, sobre Apache Arrow. Ele entra no Spark pela API de plugins e reescreve o plano físico.

- O **Rust** tira o GC do caminho e dá controle de memória; os lotes de **Arrow** tiram a conversão linha↔coluna.
- O que ele acelera são operadores: **scan, filter, project, hash aggregate, exchange, shuffle, sort, join**.
- Seu código não sabe que ele existe — DataFrame, SQL e o JSON do pipeline seguem idênticos.
- Onde não há equivalente nativo, o Spark original executa. Nada quebra por falta de suporte.

Onde exatamente ele se encaixa.

1 · Sua query — DataFrame, SQL ou o JSON do Sparquet

Nada muda aqui. É a mesma API.



2 · Catalyst: análise, otimização, plano físico

Spark puro. Comet não toca no otimizador.



3 · CometPlugin: regras sobre o plano físico

Percorre a árvore e troca cada operador suportado pelo equivalente nativo — FileSourceScan vira CometNativeScan, HashAggregate vira CometHashAggregate.



4 · JNI: o executor chama o nativo

O ramo convertido vira um plano do DataFusion do outro lado da fronteira. Os dados atravessam como buffers Arrow em memória off-heap — sem serializar, sem copiar.



5 · Execução em Rust, colunar, vetorizada

Lotes de Arrow, sem GC, sem objeto por linha.

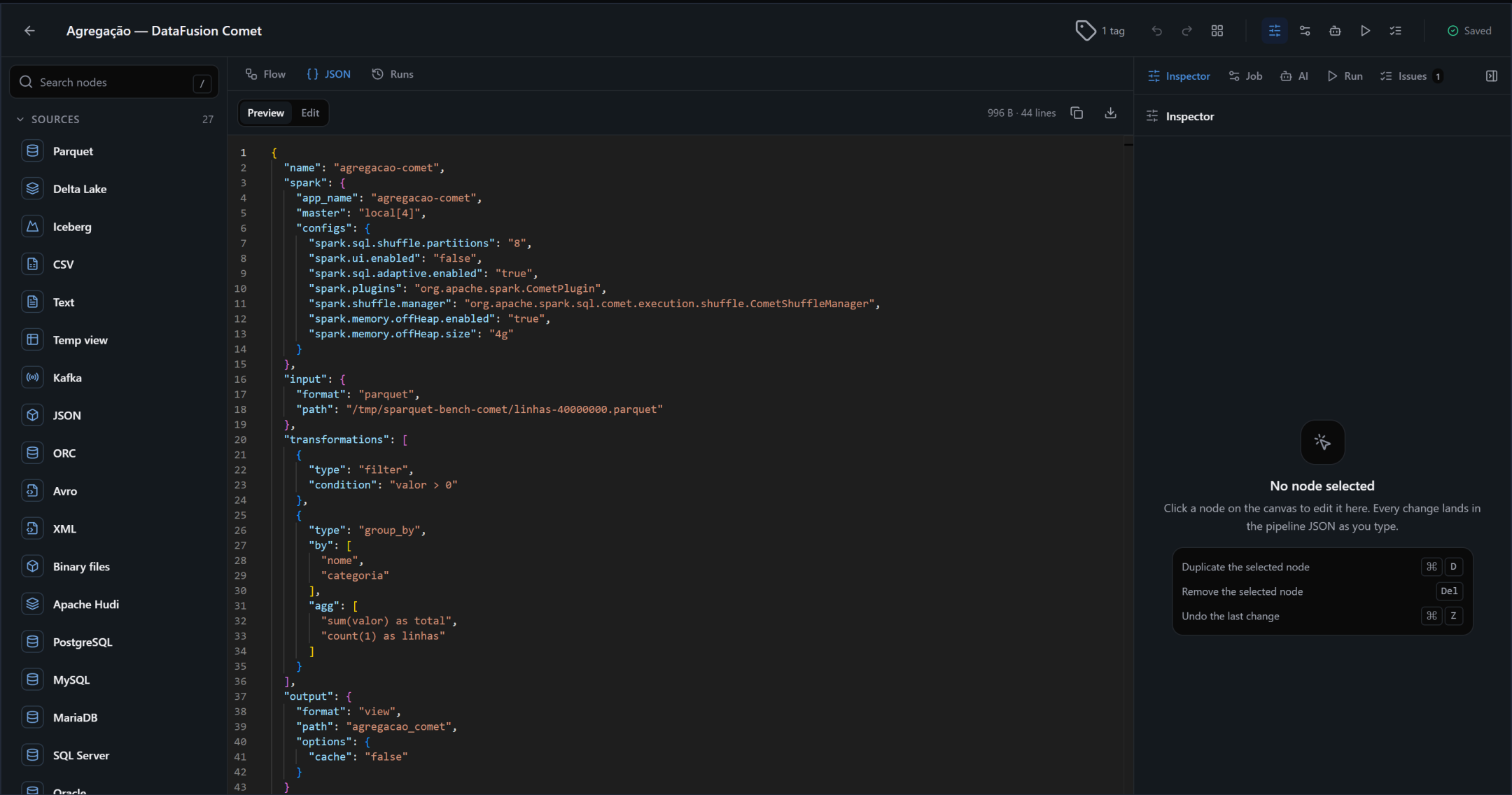


6 · CometColumnarToRow na borda

Só onde a JVM precisa das linhas de volta: o `collect`, um operador não suportado, a escrita.

Quatro configurações. Zero mudança no pipeline.

Os dois lados do teste são **o mesmo JSON**: mesma fonte, mesmo filtro, mesmo `group_by`, mesma saída. A única diferença são estas quatro linhas em `spark.configs`.



`spark.plugins` carrega o plugin; o shuffle manager nativo evita voltar para a JVM no embaralhamento; o off-heap é obrigatório — os operadores nativos alocam fora do heap.

O jar (88 MB) precisa estar no `--driver-class-path` antes de a JVM subir: `spark.jars` é resolvido tarde demais para carregar a classe do plugin.

A conversão é por operador.

Comet não é um interruptor liga/desliga da query inteira. Cada nó do plano é avaliado sozinho: **o que ele sabe executar vira nativo, o resto continua na JVM** — e o plano final costuma ser híbrido.

Isso tem custo: toda fronteira entre nativo e JVM paga uma conversão colunar→linha. Um plano muito picotado pode **perder** o ganho nessas bordas. Por isso o número depende da forma da consulta — e por isso a página 09 mostra o plano, não só o cronômetro.

- Expressão sem equivalente nativo — uma UDF em Python, por exemplo — **não** converte.
- Tipos e funções cobertos crescem a cada versão; o que não é coberto cai para o Spark.
- Nada falha por falta de suporte. O pior caso é executar como executava antes.

O que foi medido, e como.

- **40.000.000 de linhas**, 290 MB em Parquet, 16 arquivos — geradas uma vez, em processo próprio e sem Comet.
- **Duas JVMs**, uma por configuração: `spark.plugins` só vale na criação da sessão.
- **Execução de aquecimento descartada** — a primeira paga JIT, metadado de Parquet e page cache frio.
- **Mediana de 3 medições**, não média: em máquina compartilhada, média com N pequeno mede o vizinho.
- A cronometragem termina em uma **ação** (`collect` / `count`). Sem isso o Spark é preguiçoso e a medição viraria planejamento.
- O cache do writer foi **desligado**: com ele, a segunda repetição leria memória e mediria o cache do Spark.

Intel i7-11700KF · 16 vCPU · 16 GB · WSL2 (Linux 6.6) · Java 17 (Temurin 17.0.20.1) · PySpark 4.1.1 · Comet 1.0.0 (`spark4.1_2.13`) · `master=local[4]` · `shuffle.partitions=8` · AQE ligado · off-heap 4 GB.

RESULTADO

2,52x na agregação.
1,38x na varredura.

FORMA DA CONSULTA	SEM COMET	COM COMET	GANHO
Filtro + group_by com duas agregações	4,33 s	1,72 s	2,52x
Filtro seletivo + contagem	1,05 s	0,76 s	1,38x

O ganho **não é um número único**. Quanto mais trabalho de CPU por linha — agregação, hash, shuffle — mais o nativo aparece. Uma varredura já limitada por I/O ganha menos, e mostrar as duas pontas é o mínimo honesto.

Medianas de 3 repetições cronometradas, após aquecimento descartado. Mínimos: 4,23 s / 1,64 s e 1,01 s / 0,75 s.

PROVA

Número sem plano não vale nada.

Aceleração medida só significa alguma coisa se o operador era mesmo nativo. Estes são os nós que apareceram no plano físico **da execução cronometrada**:

CometNativeScan

CometFilter

CometHashAggregate

CometExchange

CometNativeShuffle

Agregação: leitura, filtro, agregação e embaralhamento — a cadeia inteira em execução nativa.

CometNativeScan

CometProject

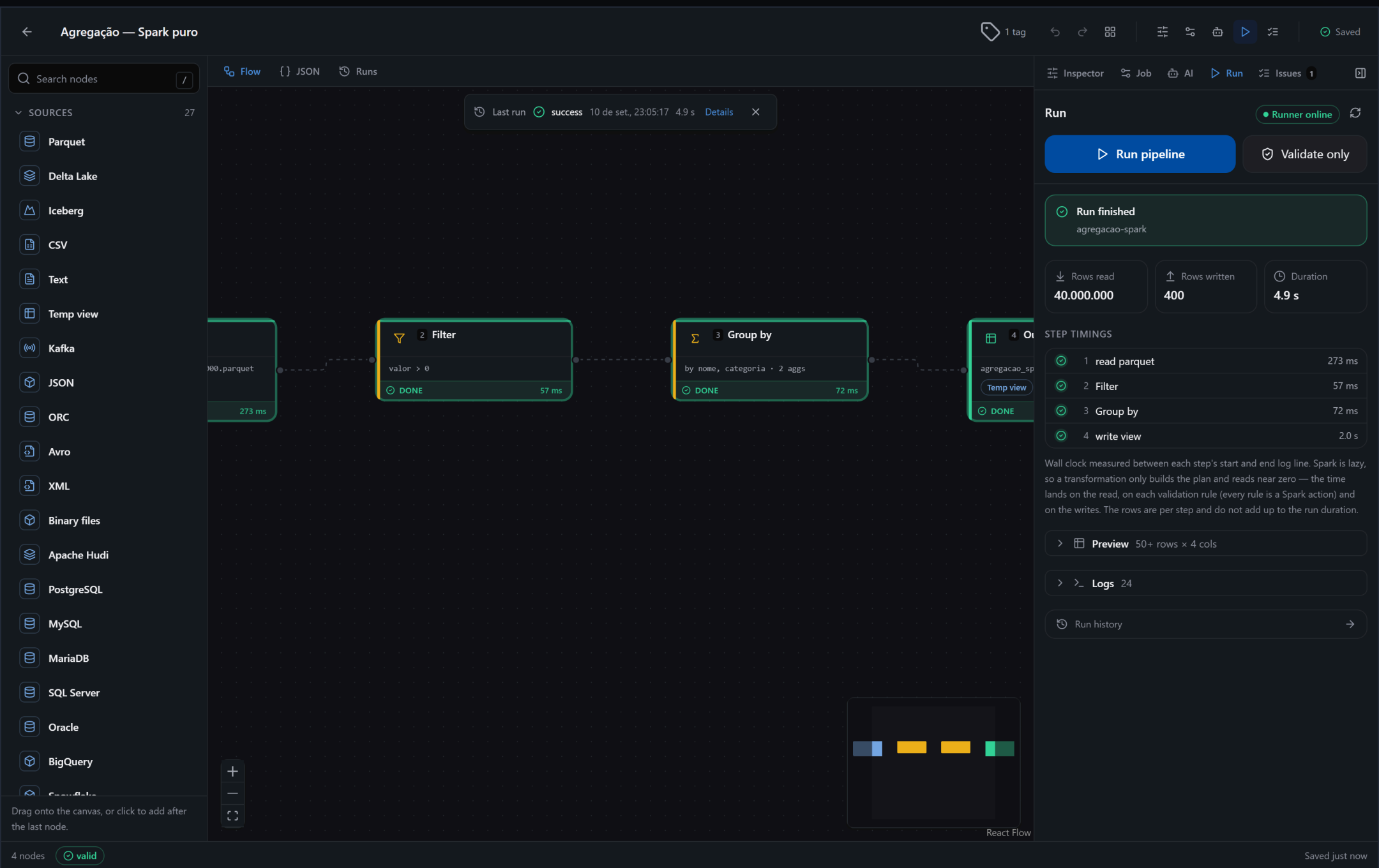
CometFilter

CometColumnarToRow

Varredura: o último nó é a fronteira — a contagem volta para a JVM. É exatamente a borda que separa o 2,52× do 1,38×.

4,9 s para agregar 40 milhões de linhas.

O mesmo teste, agora executado pelo **Sparquet Studio**: o Job é o pipeline JSON, o runner é um processo Spark local, e o painel mostra linhas lidas, duração e o tempo de cada passo.



Spark 4.1.1 puro. O tempo se concentra na ação final: ler, filtrar e agregar as 40 milhões de linhas.

NO STUDIO · COM COMET

2,4 s. Mesmo Job, mesmos dados.

Outro runner, com o jar do Comet no classpath do driver e as quatro configurações ligadas. Mesmo canvas, mesmo JSON, as mesmas **40.000.000** de linhas lidas e as mesmas 400 escritas.

←

Agregação — DataFusion Comet

1 tag

↶

↷

⌵

⌵

⌵

▶

⌵

🟢 Saved

🔍 Search nodes

/

▼ SOURCES

27

Parquet

Delta Lake

Iceberg

CSV

Text

Temp view

Kafka

JSON

ORC

Avro

XML

Binary files

Apache Hudi

PostgreSQL

MySQL

MariaDB

SQL Server

Oracle

BigQuery

CloudStorage

Drag onto the canvas, or click to add after the last node.

🔗 Flow

{ } JSON

🕒 Runs

🕒 Last run

🟢 success

10 de set., 23:05:44

2.4 s

Details

×

📄 100.parquet

265 ms

2 Filter

valor > 0

🟢 DONE

56 ms

3 Group by

by nome, categoria · 2 aggs

🟢 DONE

72 ms

4 On write

agregacao_comet

Temp view

🟢 DONE

+

−

↻

React Flow

Inspector

Job

AI

▶ Run

⌵ Issues 1

🔗

Run

🟢 Runner online

↻

▶ Run pipeline

🛡️ Validate only

🟢 Run finished

agregacao-comet

📄 Rows read

40.000.000

📄 Rows written

400

🕒 Duration

2.4 s

STEP TIMINGS

🟢 1 read parquet

265 ms

🟢 2 Filter

56 ms

🟢 3 Group by

72 ms

🟢 4 write view

833 ms

Wall clock measured between each step's start and end log line. Spark is lazy, so a transformation only builds the plan and reads near zero — the time lands on the read, on each validation rule (every rule is a Spark action) and on the writes. The rows are per step and do not add up to the run duration.

> 📄 Preview

50+ rows × 4 cols

> >_ Logs

24

🕒 Run history

→

4 nodes

🟢 valid

Saved just now

Metade do tempo de parede — e o resultado idêntico, linha por linha.

DataFusion Comet no Sparquet Studio

11

REPETIÇÃO

Não foi uma execução de sorte.

O Studio guarda o histórico de cada Job. Cinco execuções de cada lado, mesma máquina, mesmo arquivo: **mediana de 4,9 s sem Comet** contra **2,5 s com Comet**.

←

Agregação — Spark puro

1 tag

↶

↷

⌵

⌵

⌵

⌵

▶

⌵

🟢 Saved

🔍 Search nodes

/

▼ SOURCES

27

Parquet

Delta Lake

Iceberg

CSV

Text

Temp view

Kafka

JSON

ORC

Avro

XML

Binary files

Apache Hudi

PostgreSQL

MySQL

MariaDB

SQL Server

Oracle

BigQuery

Snowflake

Drag onto the canvas, or click to add after the last node.

🟢 valid

🔍 Flow

{ } JSON

🕒 Runs

Agregação — Spark puro

5 executions, most recent first

Success rate

100%

5 succeeded, 0 failed

Median duration

4.9 s

Last run

just now

All 5

Running 0

Failed 0

Succeeded 5

Status

Run id

Started

Duration

Run as

Launched

🟢 success

f08d010c1620412b88099eae2899cb52

just now

4.9 s

pasqualini

Manually

🟢 success

c9c9220702e04d51819bc9fa5061d504

2 min ago

5.0 s

pasqualini

Manually

🟢 success

434af84268504251a42989f77c738e24

8 min ago

5.9 s

pasqualini

Manually

🟢 success

21c7482a4c59485ebf8b7a78f7298ff8

19 min ago

4.6 s

pasqualini

Manually

🟢 success

8a91ccf266354e44ac85cfe2bd556a79

19 min ago

4.8 s

pasqualini

Manually

🔄 Refresh

🔍 Inspector

⌵ Job

📦 All

▶ Run

⌵ Issues 1

📄

Run

🟢 Runner online

🔄

▶ Run pipeline

🛡️ Validate only

🟢 Run finished

agregacao-spark

⬇️ Rows read

40.000.000

⬆️ Rows written

400

🕒 Duration

4.9 s

STEP TIMINGS

🟢 1 read parquet

273 ms

🟢 2 Filter

57 ms

🟢 3 Group by

72 ms

🟢 4 write view

2.0 s

Wall clock measured between each step's start and end log line. Spark is lazy, so a transformation only builds the plan and reads near zero — the time lands on the read, on each validation rule (every rule is a Spark action) and on the writes. The rows are per step and do not add up to the run duration.

▶ 📄 Preview

50+ rows × 4 cols

▶ >_ Logs

24

🕒 Run history

→

4 nodes

🟢 valid

Saved just now

Sem Comet: 4,6 s · 4,8 s · 5,9 s · 5,0 s · 4,9 s. Com Comet as cinco ficaram entre 2,4 s e 2,5 s — a variação também encolhe quando o GC sai do caminho.

DataFusion Comet no Sparquet Studio

12

O QUE O NÚMERO NÃO MOSTRA

O custo de ligar.

4 GB

de off-heap dimensionado só para os operadores nativos — memória que a execução sem plugin não usava.

88 MB

de jar para distribuir, e ele precisa estar no classpath do driver antes de a JVM subir.

Linux

a biblioteca nativa é publicada para Linux. Em outra plataforma o plugin se desativa em silêncio — e você mede Spark puro achando que mediu Comet.

Híbrido

plano muito picotado paga conversão em cada borda. Confira sempre os nós Comet no plano.

Vale quando a carga é **limitada por CPU** e repetitiva: agregação pesada, shuffle grande, varredura de Parquet em volume. Em job pequeno, o ganho se perde na partida da sessão.

FECHANDO

O mesmo pipeline. Metade do tempo.

- Comet acelera a **execução**; ele não reescreve a sua engenharia de dados.
- Liga por configuração, cai para o Spark quando não sabe, e devolve o mesmo resultado.
- O ganho medido aqui: **2,52×** na agregação, **1,38×** na varredura, **~2×** no Job de ponta a ponta.
- Todo o teste — geração dos dados, as duas JVMs, o plano e os tempos — está no repositório.

```
# benchmark reproduzível  
$ SPARQUET_IT_COMET_JAR=/caminho/comet.jar \  
python tests/io/integration/bench_comet.py
```

Sparquet · github.com/VictorPasqualini/sparquet

Documentação · sparquet.web.app

Apache DataFusion Comet · datafusion.apache.org/comet